A Single Foundation Model for Speed-Test Early Termination

Jithendra (2025MCS2743) Nitin (2025MCS2152)
FMFSTLT Course Project

Abstract

TurboTest-style early termination uses two task-specific models: an XGBoost regressor that predicts final throughput from a prefix, and a Transformer policy that decides when a test can stop safely. We investigate whether a single foundation model pretrained on speed-test traces can replace this two-model pipeline while preserving the same Stage 1 to Stage 2 decision flow. Our final system uses a causal bucket-level encoder, per-decision throughput heads, and a policy module that consumes the foundation model's own Stage 1 predictions. On a 12-sampled-day public M-Lab dataset spanning April 2024 to March 2025, the foundation policy matches the specialized policy closely, but the deployed system does not fully beat TurboTest because XGBoost remains stronger at prefix throughput prediction.

1 Problem and Data

TurboTest-style early termination uses Stage 1 prefix throughput prediction followed by a Stage 2 stop policy. We ask whether one pretrained foundation model can replace the separate XGBoost regressor and Transformer policy while preserving this order. We use public M-Lab-derived traces represented as 100 buckets at 100 ms resolution with 13 normalized features per bucket. The data follows a sampled-day scale: 10 train/test dates from April 2024 to January 2025 and two robustness dates from February and March 2025. The training source has 800k tests, split into 720k train UUIDs and 80k validation UUIDs; test and robustness contain about 40k and 132k tests. Baseline comparisons are frozen at $\epsilon = 10\%$ because other epsilon Stage 2 datasets were found to contain stale XGBoost predictions.

2 Models

TurboTest reproduction. Stage 1 flattens the latest 20×13 bucket window to 260 features and trains an XGBoost regressor. Stage 2 uses 20 decision points at 500 ms stride, computes a permanent-safe suffix oracle from Stage 1 errors, and trains an 8-layer Transformer stop policy. The frozen $\epsilon = 10\%$ baseline uses threshold 0.25.

Foundation model. The final foundation architecture keeps the same flow:

$x \in \mathbb{R}^{100\times 13} \rightarrow$ causal bucket encoder $\rightarrow$ per-decision throughput heads $\rightarrow$ Stage 1 predictions $\rightarrow$ causal Stage 2 policy $\rightarrow$ stop probability.

The encoder is an FMNet-v3 causal Transformer with 10 layers, $d = 256$, 8 heads, and 1024 feed-forward width, initialized from causal next-bucket pretraining. Decision states are gathered at buckets $4, 9, \ldots, 99$. Stage 1 emits μ_d and log variance. Stage 2 receives only deployable features: μ_d, log variance, elapsed time, observed-bucket count, and the decision hidden state. Labels such as relative error and stop labels are not policy inputs.

Table 1: Stage 1 throughput error. Prefix MAE is averaged over valid decision points; lower is better.

Subset	XGB prefix MAE	Found. prefix MAE	Found. final MAE
Val	18.98	22.96	9.68
Test	17.72	21.06	8.56
Robust	32.14	49.61	35.21

3 Training and Evaluation

The foundation run is trained in phases. Phase 1 trains the encoder and Stage 1 heads using per-decision log-throughput MSE and final-throughput MSE. The final run uses cosine learning-rate decay and exponential moving average weights; this stabilized a previously noisy throughput trajectory. Phase 2 freezes the encoder and Stage 1 heads, detaches the Stage 1 outputs at the Stage 2 boundary, and trains only the policy module using stop-label BCE. Phase 3 end-to-end tuning was attempted but discarded because it improved policy metrics slightly while increasing Stage 1 error by about 3–4×.

We report two kinds of Stage 2 metrics. *Policy* metrics use the stored XGBoost prediction error to ask whether the chosen stop point would be safe under the TurboTest prediction source. *Deployed* metrics use the value the system would actually report at runtime. For TurboTest this is XGBoost; for the foundation system this is the foundation prefix prediction. This distinction is important because policy-only evaluation can hide a weak Stage 1 predictor.

4 Results

Table 1 shows that the foundation model learns a strong full-trace throughput head, but TurboTest's XGBoost regressor remains better at the per-decision prefix predictions that matter at early stopping time.

Table 2 separates policy quality from deployed user-facing accuracy. F+X evaluates the foundation stop policy using the same XGBoost prefix predictions used by TurboTest; this isolates the stop-policy decision. F+F is the deployed foundation system: the foundation chooses the stop point and reports its own prefix throughput prediction.

The foundation policy is tied with the specialized Stage 2 Transformer when both use the same XGBoost prediction source:

Table 2: Stage 2 early-stop results at the selected thresholds. Within is the fraction of stopped tests within 10% of final throughput.

Subset	System	F1	Policy within	Deployed within	Savings ms
Val	TurboTest B+X	0.8696	0.6684	0.6684	5241
Val	Foundation	0.8703	0.6650	0.5867	5087
Test	TurboTest B+X	0.8608	0.6786	0.6786	5145
Test	Foundation	0.8604	0.6734	0.5908	4968
Robust	TurboTest B+X	0.8565	0.6608	0.6608	5245
Robust	Foundation	0.8552	0.6598	0.5785	5073

Table 3: Run 3 diagnostic at $\epsilon = 10\%$. F+X isolates policy quality; F+F is the real deployed foundation system.

Subset	F+F deployed	F+X policy-only	B+X TurboTest
Val	0.5867	0.6650	0.6684
Test	0.5908	0.6734	0.6786
Robust	0.5785	0.6598	0.6608

F1 differs by at most 0.0013 and policy-within by at most 0.0052. However, deployed foundation accuracy loses about 8 percentage points at $\epsilon = 10\%$ because its own prefix predictions are noisier than XGBoost's at the selected stop. It also saves about 150–180 ms less.

5 What Worked and What Failed

The positive result is that the foundation architecture can learn the Stage 2 stopping behavior. The policy module is small compared with the encoder, yet it reaches essentially the same stop-policy quality as the specialized Transformer when evaluated under the same prediction source. This supports the idea that self-supervised trace representations can transfer to network-measurement control tasks.

The negative result is equally important: the foundation model does not yet replace XGBoost as the deployed Stage 1 prefix regressor. Full-trace throughput prediction becomes strong, but early-stop deployment depends on predictions made before the test finishes. At those intermediate decision points, the specialized XGBoost window model remains better. Therefore, a claim based only on final throughput MAE would be misleading for early termination.

6 Discussion and Conclusion

The final result is not a strict replacement win, but it is informative. The single foundation model preserves the Stage 1 to Stage 2 structure and learns a competitive stop policy. The bottleneck is per-decision prefix throughput prediction, not stop-policy learning. We conclude that foundation pretraining is useful for shared representation learning and policy transfer, but it does not yet fully replace TurboTest's specialized XGBoost prefix regressor.

For future work, the most direct next experiment is teacher distillation from the stored XGBoost prefix predictions during Stage 1 training. XGBoost would be used only as a training-time teacher, while deployment would still call only the foundation model. We stop experiments here and submit the working code with this revised interpretation.